Анализ данных с использованием языка Python

Пояснения к коду. Семинар 15

Разведочный анализ данных (EDA): структура отчёта и полный пример

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-15-eda.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда. Новых приёмов в ноутбуке почти нет: отчёт собран из функций и команд семинаров 8–13. Поэтому подробно разобрана библиотека функций, ячейка 1: как устроена функция Python и что делает каждая из 32 функций. В остальных ячейках названы вызовы функций библиотеки и команды, которые легко не понять с первого взгляда. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочниках `reference/pandas.md`, `reference/visualization.md` и `reference/statistics.md`.

1 Библиотека функций

Как читать функцию. Функция — кусок кода с именем, который выполняется при каждом вызове с новыми значениями параметров. В библиотеке встречаются такие конструкции Python.

- `def count_line(periods, title, xlabel, ax=None):` — заголовок функции: имя, параметры в скобках и двоеточие. Тело функции — строки с отступом под заголовком. `return` — что функция отдаёт тому, кто её вызвал.
- `ax=None`, `factor=1.5`, `alpha=0.05` — значения по умолчанию: при вызове такой параметр можно не указывать.
- `if ax is None:` и под ним `fig, ax = plt.subplots(...)` — если оси не переданы, функция создаёт свою фигуру, а если переданы, рисует на них. Так одна функция рисует и отдельный график, и график на заранее заданных осях: в ячейке 23 функции `mean_bars` передают оси шире обычных.
- `q1, q3 = values.quantile([0.25, 0.75])` — распаковка: два значения в две переменные. Так же функция возвращает несколько чисел через запятую, `return result.statistic, result.pvalue, d`, а в ячейке их раскладывают: `t, p, d = t_two(women, men)`.
- `[flag_outliers(df[column], method).sum() for column in columns]` — генератор списка: значение для каждого столбца из списка, цикл `for` в одной строке.
- `raise ValueError(f'...')` — остановить функцию с сообщением об ошибке, когда параметр недопустим. `f'неизвестное правило: {method}'` — f-строка: на место `{method}` подставляется значение параметра; `{kind!r}` — значение в кавычках, чтобы было видно, что передана строка.
- `'отвергаем' if p < alpha else 'не отвергаем'` — выбор из двух значений в одной строке.
- `not 0 <= p <= 1` — двойное сравнение с отрицанием: верно, когда p лежит вне отрезка от 0 до 1.
- `*samples` — звёздочка раскладывает список на отдельные аргументы: `f_oneway(*samples)` для трёх групп — то же, что `f_oneway(первая, вторая, третья)`.
- `for ax, column in zip(axes[0], columns):` — `zip` ставит в пары оси и столбцы, цикл берёт по паре.

Функции не меняют таблицы, которые им передают: новые столбцы и маски создаются внутри функции.

Ячейка 1. Библиотека: подключение и 32 функции

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px
import seaborn as sns
from scipy import stats

sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию


def count_line(periods, title, xlabel, ax=None):  # число ДТП по периодам
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    counts = periods.value_counts().sort_index()
    counts.plot(ax=ax, marker='o')
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Число ДТП')
    return ax


def top_bars(values, n, title, ax=None):  # самые частые значения
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    counts = values.value_counts().head(n)[::-1]
    counts.plot.barh(ax=ax)
    ax.set_title(title)
    ax.set_xlabel('Число ДТП')
    return ax


def share_bars(rows, columns, order, title, ax=None):  # состав групп долями
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    shares = pd.crosstab(rows, columns, normalize='index')[order]
    shares.plot(kind='bar', stacked=True, rot=0, ax=ax)
    ax.set_title(title)
    ax.set_ylabel('Доля ДТП')
    return ax


def count_hist(values, title, xlabel, ax=None):  # гистограмма целых значений
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    sns.histplot(x=values, discrete=True, ax=ax)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Число ДТП')
    return ax


def box_by(df, value, by, order, title, ax=None):  # ящики по группам
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    sns.boxplot(data=df, x=by, y=value, order=order, ax=ax)
```

```
    ax.set_title(title)
    return ax


def density_by(df, column, group, title, xlabel, ax=None):  # плотности по группам
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    sns.kdeplot(data=df, x=column, hue=group, common_norm=False, ax=ax)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Плотность')
    return ax


def corr_heat(df, columns, title, ax=None):  # тепловая карта корреляций
    if ax is None:
        fig, ax = plt.subplots(figsize=(6, 5))
    corr = df[columns].corr()
    sns.heatmap(corr, cmap='coolwarm', center=0, annot=True, fmt='.2f', ax=ax)
    ax.set_title(title)
    return ax


def share_line(df, period, flag, title, xlabel, ax=None):  # доля по периодам
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    sns.lineplot(data=df, x=period, y=flag, ax=ax)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Доля')
    return ax


def mean_bars(df, group, value, title, xlabel, ylabel, ax=None):  # среднее по группам
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    sns.barplot(data=df, x=group, y=value, ax=ax)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    return ax


def hist_panels(df, column, by, title):  # гистограммы на панелях
    g = sns.displot(data=df, x=column, col=by, col_wrap=4, discrete=True, height=2.5)
    g.figure.suptitle(title)
    return g.figure


def points_map(df, box, color, hover, title):  # точки на карте
    lat_min, lat_max, lon_min, lon_max = box
    inside = df[df['lat'].between(lat_min, lat_max) & df['lon'].between(lon_min, lon_max)]
    fig = px.scatter_map(inside, lat='lat', lon='lon', color=color, hover_data=hover, zoom=7, height=600, title=title)
    return fig

```

```python

def missing_bars(df, title, ax=None):  # доли пропусков по столбцам
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 4))
    shares = df.isna().mean()
    shares = shares[shares > 0].sort_values()
    shares.plot.barh(ax=ax)
    ax.set_title(title)
    ax.set_xlabel('Доля пропусков')
    return ax


def iqr_bounds(values, factor=1.5):  # границы IQR
    q1, q3 = values.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - factor * iqr, q3 + factor * iqr


def flag_outliers(values, method='iqr'):  # маска выбросов
    if method == 'iqr':
        lower, upper = iqr_bounds(values)
        return (values < lower) | (values > upper)
    if method == 'z':
        z = (values - values.mean()) / values.std()
        return z.abs() > 3
    if method == 'percentile':
        low, high = values.quantile([0.01, 0.99])
        return (values < low) | (values > high)
    raise ValueError(f'неизвестное правило: {method}')


def outlier_table(df, columns):  # выбросы по трём правилам
    table = pd.DataFrame(index=columns)
    for method in ['iqr', 'z', 'percentile']:
        table[method] = [flag_outliers(df[column], method).sum() for column in columns]
    return table


def missing_by_group(df, column, group):  # доля пропусков по группам
    shares = df[column].isna().groupby(df[group]).mean()
    return shares.sort_values(ascending=False)


def logic_checks(df):  # логические проверки ДТП
    victims = df['dead_count'] + df['injured_count']
    return pd.Series({
        'пострадавших больше участников': (victims > df['participants_count']).sum(),
        'машин больше участников': (df['vehicles_count'] > df['participants_count']).sum(),
        'машин нет': (df['vehicles_count'] == 0).sum(),
        'участников нет': (df['participants_count'] == 0).sum(),
    })


def duplicate_report(df, group):  # повторы по группам
    repeated = df.duplicated(keep=False)
    report = pd.DataFrame({
        'строк': df.groupby(group).size(),
        'в повторах': repeated.groupby(df[group]).sum(),
```

```
    })
    return report.sort_values('в повторах', ascending=False)


def normality_table(df, columns, alpha=0.05):  # тест Д'Агостино по столбцам
    table = pd.DataFrame(index=columns)
    table['n'] = [df[column].count() for column in columns]
    table['асимметрия'] = [df[column].skew() for column in columns]
    table['p'] = [stats.normaltest(df[column].dropna()).pvalue for column in columns]
    table['решение'] = np.where(table['p'] < alpha, 'отвергаем', 'не отвергаем')
    return table


def normality_report(df, columns, alpha=0.05):  # нормальность и выбор теста
    report = normality_table(df, columns, alpha)
    small = np.where(report['решение'] == 'отвергаем', 'непараметрический тест',
                     'мало данных: решать по Q-Q графику и асимметрии')
    report['что дальше'] = np.where(report['n'] >= 30, 't-тест', small)
    return report


def qq_plot(values, title, ax=None):  # Q-Q график
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
    stats.probplot(values.dropna(), plot=ax)
    ax.set_title(title)
    ax.set_xlabel('Квантили нормального распределения')
    ax.set_ylabel('Значения')
    return ax


def qq_panels(df, columns):  # Q-Q графики рядом
    fig, axes = plt.subplots(1, len(columns), figsize=(4 * len(columns), 4), squeeze=False)
    for ax, column in zip(axes[0], columns):
        qq_plot(df[column], column, ax=ax)
    fig.tight_layout()
    return fig


def decision(p, alpha=0.05):  # решение словами
    if not 0 <= p <= 1:
        raise ValueError(f'p-значение должно лежать от 0 до 1, получено {p}')
    return 'отвергаем' if p < alpha else 'не отвергаем'


def ci_mean(values, conf=0.95):  # интервал среднего
    values = values.dropna()
    return stats.t.interval(conf, len(values) - 1, loc=values.mean(), scale=stats.sem(values))


def bonferroni(pvalues, alpha=0.05):  # поправка Бонферрони
    return pvalues < alpha / len(pvalues)


def t_two(a, b):  # t Уэлча и d Коэна
    a, b = a.dropna(), b.dropna()
```

```
225     result = stats.ttest_ind(a, b, equal_var=False)
226     d = (a.mean() - b.mean()) / np.sqrt((a.var() + b.var()) / 2)
227     return result.statistic, result.pvalue, d
228 
229 
230 def mw_two(a, b):  # критерий Манна-Уитни
231     result = stats.mannwhitneyu(a.dropna(), b.dropna())
232     return result.statistic, result.pvalue
233 
234 
235 def chi_square(df, a, b):  # хи-квадрат и V Крамера
236     table = pd.crosstab(df[a], df[b])
237     result = stats.chi2_contingency(table)
238     plain = stats.chi2_contingency(table, correction=False)
239     v = np.sqrt(plain.statistic / (table.to_numpy().sum() * ↩
↪ (min(table.shape) - 1)))
240     return result.statistic, result.pvalue, result.dof, v
241 
242 
243 def corr_test(x, y):  # корреляции Пирсона и Спирмена
244     both = x.notna() & y.notna()
245     pearson = stats.pearsonr(x[both], y[both])
246     spearman = stats.spearmanr(x[both], y[both])
247     return pearson.statistic, pearson.pvalue, spearman.statistic, ↩
↪ spearman.pvalue
248 
249 
250 def groups_test(df, value, group):  # ANOVA Уэлча и Крускал-Уоллис
251     samples = [values.dropna() for name, values in df.groupby(group)[value]]
252     anova = stats.f_oneway(*samples, equal_var=False)
253     kruskal = stats.kruskal(*samples)
254     return pd.Series({'F': anova.statistic, 'p ANOVA': anova.pvalue, 'H': ↩
↪ kruskal.statistic, 'p Крускала': kruskal.pvalue})
255 
256 
257 def effect_label(value, kind):  # размер эффекта словом
258     thresholds = {'d': (0.2, 0.5, 0.8), 'V': (0.1, 0.3, 0.5), 'r': (0.1, ↩
↪ 0.3, 0.5)}
259     if kind not in thresholds:
260         raise ValueError(f"kind должен быть 'd', 'V' или 'r', получено ↩
↪ {kind!r}")
261     if np.isnan(value):
262         raise ValueError('размер эффекта - nan: в данных пропуски')
263     small, medium, large = thresholds[kind]
264     size = abs(value)
265     if size >= large:
266         return 'большой'
267     if size >= medium:
268         return 'средний'
269     if size >= small:
270         return 'малый'
271     return 'ничтожный'
272 
273 
274 def hypotheses_report(df, drivers):  # таблица гипотез курса
275     experience, health, gender = drivers['experience_clean'], ↩
↪ drivers['health'], drivers['gender']
276     t1, p1, d1 = t_two(experience[health == 'Погиб'], experience[health == ↩
↪ 'Ранен'])
277     year = df['datetime'].dt.year
278     by_year = pd.crosstab(df['district'], year)
```

```
279     by_year = by_year[~by_year.index.str.contains('область')]
280     paired = stats.ttest_rel(by_year[year.min()], by_year[year.max()])
281     flags = df.assign(dark=df['light'].str.contains('темное'), ↩
↪ fatal=df['dead_count'] > 0, defects=df['road_defects'] > 0)
282     chi3, p3, dof3, v3 = chi_square(flags, 'dark', 'fatal')
283     r4, p4, rho4, p_rho4 = corr_test(experience, drivers['vehicle_age'])
284     t5, p5, d5 = t_two(experience[gender == 'Женский'], experience[gender ↩
↪ == 'Мужской'])
285     chi6, p6, dof6, v6 = chi_square(flags, 'defects', 'fatal')
286     rows = [
287         ('стаж погибших и раненых водителей', 't Уэлча', ↩
↪ experience[health.isin(['Погиб', 'Ранен'])].count(), t1, p1, d1, ↩
↪ effect_label(d1, 'd')),
288         ('ДТП по районам в первый и последний год', 'парный t', ↩
↪ len(by_year), paired.statistic, paired.pvalue, np.nan, '-'),
289         ('тёмное время и ДТП с погибшими', 'хи-квадрат', len(df), chi3, ↩
↪ p3, v3, effect_label(v3, 'V')),
290         ('стаж и возраст машины', 'Пирсон', (experience.notna() & ↩
↪ drivers['vehicle_age'].notna()).sum(), r4, p4, r4, effect_label(r4, 'r')),
291         ('стаж женщин и мужчин за рулём', 't Уэлча', ↩
↪ experience[gender.isin(['Женский', 'Мужской'])].count(), t5, p5, d5, ↩
↪ effect_label(d5, 'd')),
292         ('недостатки дороги и ДТП с погибшими', 'хи-квадрат', len(df), ↩
↪ chi6, p6, v6, effect_label(v6, 'V')),
293     ]
294     table = pd.DataFrame(rows, columns=['вопрос', 'тест', 'n', ↩
↪ 'статистика', 'p', 'эффект', 'размер'], index=range(1, 7))
295     table['решение'] = np.where(table['p'] < 0.05, 'отвергаем', 'не ↩
↪ отвергаем')
296     table['после поправки'] = np.where(table['p'] < 0.05 / len(table), ↩
↪ 'отвергаем', 'не отвергаем')
297     return table
```

– Строки 1–8 — библиотеки и оформление графиков, как на семинарах 8–13.

– `count_line`, строки 11–19, — `periods.value_counts().sort_index()`: число ДТП в каждом периоде в порядке периодов; `plot(ax=ax, marker='o')` — линия с точками.

– `top_bars`, 22–29, — `value_counts().head(n)`: n самых частых значений; `[::-1]` — обратный порядок: `barh` рисует первую строку внизу, а так самый частый столбец оказывается сверху.

– `share_bars`, 32–39, — `pd.crosstab(rows, columns, normalize='index')`: доли внутри каждой группы строк; `[order]` — столбцы таблицы в заданном порядке, это порядок слоёв; `stacked=True` — слои друг на друге, `rot=0` — подписи групп без поворота.

– `count_hist`, 42–49, — `sns.histplot(x=values, discrete=True)`: по столбцу на каждое целое значение.

– `box_by`, 52–57, — `sns.boxplot(data=df, x=by, y=value, order=order)`: ящики значения по группам в заданном порядке.

– `density_by`, 60–67, — `sns.kdeplot(..., hue=group, common_norm=False)`: кривая плотности на каждую группу; каждая нормирована внутри своей группы, поэтому график сравнивает форму, а не число.

– `corr_heat`, 70–76, — `df[columns].corr()` и `sns.heatmap`: `cmap='coolwarm'`, `center=0` — ноль белый, плюс красный, минус синий; `annot=True`, `fmt='.2f'` — числа в клетках с двумя знаками.

– `share_line`, 79–86, — `sns.lineplot(data=df, x=period, y=flag)`: среднее флага `True` или `False` в каждом периоде — это доля, а полоса — её 95-процентный интервал.

– `mean_bars`, 89–96, — `sns.barplot(data=df, x=group, y=value)`: среднее или доля по группам, усы — 95-процентный интервал.

– `hist_panels`, 99–102, — `sns.displot(..., col=by, col_wrap=4, discrete=True, height=2.5)`: по панели на группу, не больше четырёх в ряд, высота панели — 2.5 дюйма. `displot` сам создаёт фигуру, поэтому параметра `ax` у функции нет; `g.figure.suptitle(title)` — общий заголовок, функция возвращает фигуру.

– `points_map`, 105–109, — строка 106 раскладывает рамку `box` на четыре числа; `between` — точки внутри рамки; `px.scatter_map(..., hover_data=hover, zoom=7, height=600)` — карта plotly: `hover_data` — столбцы подсказки при наведении, `zoom` — масштаб, `height` — высота в пикселях.

– `missing_bars`, 112–120, — `df.isna().mean()`: доля пропусков каждого столбца; `shares[shares > 0].sort_values()` — только столбцы с пропусками, по возрастанию.

– `iqr_bounds`, 123–126, — первый и третий квартили, межквартильный размах и две границы; `factor=1.5` — множитель по умолчанию.

– `flag_outliers`, 129–139, — маска выбросов по одному из трёх правил: `'iqr'` — за границами IQR, `'z'` — дальше трёх стандартных отклонений от среднего, `'percentile'` — ниже 1-го и выше 99-го процентиля; при другом значении `method` — ошибка.

– `outlier_table`, 142–146, — `pd.DataFrame(index=columns)`: пустая таблица с названиями столбцов в строках; цикл добавляет по столбцу на правило, `sum()` маски — число выбросов.

– `missing_by_group`, 149–151, — `df[column].isna().groupby(df[group]).mean()`: маска пропусков группируется по другому столбцу той же таблицы, среднее маски — доля.

– `logic_checks`, 154–161, — четыре маски, `sum()` каждой — число ДТП с нарушением; `pd.Series({...})` — Series из словаря: ключи — метки, значения — числа.

– `duplicate_report`, 164–170, — `df.duplicated(keep=False)`: отмечает все строки каждой группы одинаковых строк, и первую тоже; `df.groupby(group).size()` — строк в группе, `repeated.groupby(df[group]).sum()` — из них в повторах.

– `normality_table`, 173–179, — для каждого столбца: `count()` — число значений без пропусков, `skew()` — асимметрия, `stats.normaltest(...).pvalue` — p теста Д'Агостино; `np.where(table['p'] < alpha, ...)` — решение словами.

– `normality_report`, 182–187, — столбец «что дальше». Первая команда `np.where` готовит ответ для малой выборки: по решению — непараметрический тест или график; вторая выбирает: при `n >= 30` — t-тест, иначе ответ для малой выборки.

– `qq_plot`, 190–197, — `stats.probplot(values.dropna(), plot=ax)`: Q–Q график на данных осях.

– `qq_panels`, 200–205, — `plt.subplots(1, len(columns), ..., squeeze=False)`: панели в один ряд, по 4 дюйма ширины на панель. С `squeeze=False` оси всегда приходят таблицей строк и столбцов, даже для одной панели, и `axes[0]` — первая строка; `fig.tight_layout()` — панели без наложения подписей.

– `decision`, 208–211, — проверка, что p от 0 до 1, и решение словами.

– `ci_mean`, 214–216, — `stats.t.interval(conf, len(values) - 1, loc=values.mean(), scale=stats.sem(values))`: интервал среднего по t-распределению; `sem` — стандартная ошибка среднего.

– `bonferroni`, 219–220, — каждое p сравнивается с `alpha / len(pvalues)`; результат — маска.

– `t_two`, 223–227, — пропуски убраны в обеих группах, t-тест Уэлча с `equal_var=False` и коэффициент d Коэна, как на семинаре 13; функция возвращает три числа.

– `mw_two`, 230–232, — критерий Манна–Уитни: статистика и p.

– `chi_square`, 235–240, — таблица сопряжённости `pd.crosstab(df[a], df[b])`.

– Строка 237 — `chi2_contingency(table)`: хи-квадрат, для таблицы 2 × 2 с поправкой Йейтса, из него p.

– Строки 238–239 — коэффициент V Крамера по статистике без поправки, `correction=False`: в формуле `table.to_numpy().sum()` — n, а `min(table.shape) - 1` — меньшее из чисел строк и столбцов минус один.

– `corr_test`, 243–247, — маска `both`: строки, где известны оба значения; корреляции Пирсона и Спирмена с p по этим строкам.

– `groups_test`, 250–254, — строка 251 — список выборок, по одной на группу: цикл по `groupby` даёт пары «имя группы, значения»; `stats.f_oneway(*samples, equal_var=False)` — ANOVA Уэлча, `stats.kruskal(*samples)` — Крускал–Уоллис.

– `effect_label`, 257–271, — словарь порогов: малый, средний и большой эффект для d, V и r; две проверки с ошибкой; `abs(value)` — знак не важен; условия проверяются от большого эффекта к малому, и первое выполненное возвращает слово.

– `hypotheses_report`, 274–297. Строка 275 — три столбца водителей в три переменные; строка 276 — гипотеза 1. Строки 277–280 — гипотеза 2: таблица «район на год» `pd.crosstab(df['district'], year)` без строки «Брянская область», `~by_year.index.str.contains('область')`, и парный t по столбцам первого и последнего года, `year.min()` и `year.max()`. Строка 281 — три флага одной командой `assign`; строки 282–285 — гипотезы 3–6. Строки 286–293 — строки будущей таблицы, каждая — кортеж из семи значений в круглых скобках; у парного t размера эффекта нет: `np.nan` и прочерк. Строка 294 — таблица из списка кортежей с названиями столбцов и номерами строк с единицы, строки 295–296 — решения при 0.05 и после поправки.

2 Загрузка и первичный осмотр

Ячейка 2. Загрузка двух таблиц

```
dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
↪ parse_dates=['datetime'])  # ДТП Брянской области
people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # ↩
↪ участники ДТП
print(dtp.shape, people.shape)
dtp.head(3)
```

– `index_col='id'` — столбец номера ДТП становится индексом таблицы.

– `parse_dates=['datetime']` — столбец даты и времени сразу читается типом даты: `dt.year` и `dt.hour` работают без `pd.to_datetime`.

– `dtp.head(3)` — первые три строки таблицы ДТП.

Ячейка 3. Размер, память и повторы

```
pd.DataFrame({
    'строк': [len(dtp), len(people)],
    'столбцов': [dtp.shape[1], people.shape[1]],
    'МБ в памяти': [round(dtp.memory_usage(deep=True).sum() / 1024 ** 2, 1), round(people.memory_usage(deep=True).sum() / 1024 ** 2, 1)],
    'повторов строк': [dtp.duplicated().sum(), people.duplicated().sum()],
}, index=['ДТП', 'участники'])  # размер, память, повторы
```

– `pd.DataFrame({...}, index=['ДТП', 'участники'])` — таблица из словаря: ключ — название столбца, значение — список из двух чисел, по одному на таблицу; `index` — названия строк.

– `memory_usage(deep=True)` — память каждого столбца в байтах; `deep=True` — считать и текст в строках, без него память текстовых столбцов занижена.

– `sum() / 1024 ** 2` — память таблицы в мегабайтах: `**` — степень, в мегабайте `1024 ** 2` байт.

– `duplicated().sum()` — число строк, целиком повторяющих одну из предыдущих.

Ячейка 4. Типы таблицы ДТП

```
pd.DataFrame({'тип': dtp.dtypes, 'уникальных значений': dtp.nunique()})  # типы таблицы ДТП
```

– `dtp.dtypes` — тип каждого столбца, `dtp.nunique()` — число разных значений без пропусков. У обеих Series в индексе названия столбцов, и `pd.DataFrame({...})` ставит их рядом.

Ячейка 5. Типы таблицы участников

```
pd.DataFrame({'тип': people.dtypes, 'уникальных значений': people.nunique()})  # типы таблицы участников
```

– То же для таблицы участников.

3 Пропущенные значения

Ячейка 6. Видимые пропуски

```
print(dtp.isna().sum()[dtp.isna().sum() > 0])  # пропуски в таблице ДТП
missing = pd.DataFrame({'пропусков': people.isna().sum(), 'доля': people.isna().mean().round(3)})  # пропуски участников
missing[missing['пропусков'] > 0].sort_values('доля', ascending=False)
```

– Строка 1 — `dtp.isna().sum()`: пропусков в каждом столбце; `[dtp.isna().sum() > 0]` — только столбцы, где они есть.

– Строка 2 — таблица числа и доли пропусков участников; `isna().mean()` — доля, потому что `True` считается единицей.

– Строка 3 — столбцы с пропусками по убыванию доли.

Ячейка 7. Скрытые пропуски: коды стажа

```
print(people['experience'].value_counts().sort_index().tail(5))  # самые ↩
↪ большие значения стажа
people['experience_clean'] = people['experience'].replace([95, 96, 99], ↩
↪ np.nan)  # коды стажа пропусками
print(people['experience'].isin([95, 96, 99]).sum(), ↩
↪ round(people['experience'].mean(), 2), ↩
↪ round(people['experience_clean'].mean(), 2))
```

– `value_counts().sort_index().tail(5)` — значения стажа по возрастанию, пять самых больших: так видны служебные коды.

– `replace([95, 96, 99], np.nan)` — три кода в пропуск; результат записан в новый столбец `experience_clean`, исходный не меняется.

– `isin([95, 96, 99]).sum()` — сколько значений заменено; рядом — средний стаж с кодами и без.

Ячейка 8. Пропуски графиком

```
missing_bars(people.drop(columns='experience'), 'Доля пропусков в таблице ↩
↪ участников')  # стаж уже без кодов
print(round(people['experience_clean'].isna().mean(), 3))  # доля ↩
↪ пропусков стажа
```

– `people.drop(columns='experience')` — таблица без исходного столбца стажа: на графике только стаж без кодов, иначе стаж был бы на нём дважды.

– Строка 2 — доля пропусков стажа после замены кодов.

Ячейка 9. Пропуски и роль участника

```
print((dtp['road_defect'].isna() == (dtp['road_defects'] == 0)).all())  # ↩
↪ пропуск - недостатки не установлены
main = people[people['role'].isin(['Водитель', 'Пассажир', 'Пешеход', ↩
↪ 'Велосипедист'])]  # четыре частые роли
missing_by_group(main, 'experience_clean', 'role').round(3)  # доля ↩
↪ пропусков стажа
```

– Строка 1 — две маски, пропуск в `road_defect` и ноль в счётчике `road_defects`, сравниваются построчно; `.all()` — `True`, если они совпали во всех строках.

– `isin(['Водитель', 'Пассажир', 'Пешеход', 'Велосипедист'])` — участники четырёх частых ролей, таблица `main`.

– `missing_by_group(main, 'experience_clean', 'role')` — доля пропусков стажа по ролям.

Ячейка 10. Таблица водителей

```
drivers = people[people['role'] == 'Водитель'].merge(dtp, ↩
↪ left_on='dtp_id', right_index=True)  # водители с их ДТП
drivers['vehicle_age'] = drivers['datetime'].dt.year - ↩
↪ drivers['vehicle_year']  # возраст машины в годах
drivers['vehicle_age'] = ↩
↪ drivers['vehicle_age'].where(drivers['vehicle_age'] <= 100)  # ↩
↪ невозможный возраст в пропуск
print(len(drivers), drivers['experience_clean'].count(), ↩
↪ drivers['vehicle_age'].count())
```

– `merge(dtp, left_on='dtp_id', right_index=True)` — к каждому водителю добавляются столбцы его ДТП: в таблице водителей номер ДТП — столбец `dtp_id`, в таблице ДТП — индекс.

– Строка 2 — возраст машины: год ДТП минус год выпуска.

– `where(drivers['vehicle_age'] <= 100)` — значения, где условие верно, остаются, остальные становятся пропуском.

– Строка 4 — число водителей и число известных значений стажа и возраста машины: `count()` пропуски не считает.

4 Описательные статистики

Ячейка 11. Счётчики ДТП и асимметрия

```
numeric = ['participants_count', 'vehicles_count', 'dead_count', ↩
↪ 'injured_count', 'road_defects']  # счётчики ДТП
summary = dtp[numeric].describe()
summary.loc['асимметрия'] = dtp[numeric].skew()  # скошенность распределения
summary.round(2)
```

– `numeric` — список пяти счётчиков ДТП, он нужен и дальше.

– `summary.loc['асимметрия'] = dtp[numeric].skew()` — к таблице `describe()` добавляется строка: присваивание по новой метке в `loc` создаёт строку.

– `summary.round(2)` — два знака после точки.

Ячейка 12. Стаж и возраст машины

```
summary = drivers[['experience_clean', 'vehicle_age']].describe()
summary.loc['асимметрия'] = drivers[['experience_clean', ↩
↪ 'vehicle_age']].skew()  # скошенность распределения
summary.round(2)
```

– То же для стажа и возраста машины водителей.

Ячейка 13. Сводка категорий

```
rows = {}
for name, values in [('category', dtp['category']), ('severity', dtp['severity']), ('light', dtp['light']),
                     ('weather', dtp['weather']), ('district', dtp['district']), ('road_defect', dtp['road_defect']),
                     ('role', people['role']), ('gender', people['gender']), ('health', people['health']),
                     ('violation', people['violation'])]:
    shares = values.value_counts(normalize=True)  # доли без пропусков
    rows[name] = {'значений': values.nunique(), 'самое частое': shares.index[0], 'его доля': round(shares.iloc[0], 3),
                  'доля трёх частых': round(shares.head(3).sum(), 3), 'редких меньше 1 %': (shares < 0.01).sum()}
pd.DataFrame.from_dict(rows, orient='index')  # сводка категорий
```

– Строки 2–5 — цикл по списку пар «название, Series»: `for name, values in [...]` кладёт название в `name`, а Series — в `values`. Series взяты из двух таблиц, поэтому они перечислены вручную.

– `value_counts(normalize=True)` — доли значений без пропусков, по убыванию.

– `shares.index[0]` — самое частое значение, `shares.iloc[0]` — его доля.

– `shares.head(3).sum()` — доля трёх самых частых значений; `(shares < 0.01).sum()` — число значений, которые встречаются реже чем в 1 проценте случаев.

– `rows[name] = {...}` — словарь словарей: на каждый столбец — словарь из пяти значений.

– `pd.DataFrame.from_dict(rows, orient='index')` — таблица из словаря словарей; `orient='index'` — внешние ключи становятся строками, внутренние — столбцами.

Ячейка 14. Период данных и ДТП по годам

```
dtp['year'] = dtp['datetime'].dt.year  # год ДТП
dtp['hour'] = dtp['datetime'].dt.hour  # час ДТП
dtp['fatal'] = dtp['dead_count'] > 0  # ДТП с погибшими
print(dtp['datetime'].min(), dtp['datetime'].max())
pd.crosstab(dtp['year'], dtp['severity'], margins=True, margins_name='всего')  # ДТП по годам и тяжести
```

– Строки 1–3 — год и час из даты и флаг ДТП с погибшими: три новых столбца таблицы ДТП, они нужны в разделах 5 и Г.

– Строка 4 — первая и последняя дата: период данных.

– `pd.crosstab(..., margins=True, margins_name='всего')` — таблица «год на тяжесть» с итогами: `margins=True` добавляет строку и столбец сумм, `margins_name` — их название.

5 Корреляции

Ячейка 15. Тепловая карта

```
corr_heat(dtp, numeric, 'Корреляции счётчиков ДТП');  # тепловая карта
```

– `corr_heat(dtp, numeric, ...)` — тепловая карта корреляций пяти счётчиков.

– Точка с запятой в конце — не печатать то, что вернула функция: без неё под графиком появится строка вида `<Axes: ...>`.

Ячейка 16. Пары по силе связи

```
corr = dtp[numeric].corr()
upper = np.triu(np.ones(corr.shape, dtype=bool), k=1)  # каждая пара один раз
pairs = corr.where(upper).stack().dropna().sort_values()  # пары по возрастанию
print(pairs[pairs.abs() > 0.7])  # сильные связи
print(round(drivers['experience_clean'].corr(drivers['vehicle_age']), 3))  # стаж и возраст машины
pairs.round(3)
```

– `np.ones(corr.shape, dtype=bool)` — таблица из `True` того же размера, что матрица; `np.triu(..., k=1)` — оставляет `True` только выше диагонали: каждая пара один раз и без корреляции столбца с самим собой.

– `corr.where(upper)` — значения выше диагонали остаются, остальные становятся пропуском; `stack()` — таблица в Series: метка — пара названий столбцов, значение — корреляция; `dropna()` убирает пропуски, `sort_values()` сортирует по возрастанию.

– `pairs[pairs.abs() > 0.7]` — сильные пары; их нет, и печатается пустая Series: `Series([], dtype: float64)`.

– `drivers['experience_clean'].corr(drivers['vehicle_age'])` — корреляция двух Series по строкам, где известны оба значения.

6 Визуализация

Ячейка 17. Гистограмма стажа

```
ax = count_hist(drivers['experience_clean'], 'Стаж водителей, попавших в ДТП', 'Стаж, лет')  # гистограмма стажа
ax.set_ylabel('Число водителей')
drivers['experience_clean'].value_counts().loc[[9, 10, 11, 19, 20, 21]]  # пики на круглых числах
```

– `ax = count_hist(...)` — функция возвращает оси, их запоминают, чтобы поправить подпись.

– `ax.set_ylabel('Число водителей')` — функция подписала ось «Число ДТП», а считает она водителей.

– `value_counts().loc[[9, 10, 11, 19, 20, 21]]` — число водителей с этими значениями стажа: метки индекса `value_counts` и есть значения стажа.

Ячейка 18. Плотности стажа по исходу

```
density_by(drivers, 'experience_clean', 'health', 'Стаж водителей по исходу ДТП', 'Стаж, лет')  # плотности трёх исходов
drivers.groupby('health')['experience_clean'].agg(['count', 'median'])  # число и медиана стажа
```

– `density_by(drivers, 'experience_clean', 'health', ...)` — плотности стажа трёх исходов.

– `groupby('health')['experience_clean'].agg(['count', 'median'])` — число значений и медиана стажа по исходу.

Ячейка 19. Самые частые виды ДТП

```
top_bars(dtp['category'].rename('вид ДТП'), 8, 'Восемь самых частых видов ДТП')  # самые частые виды
dtp['category'].value_counts().head(8)
```

– `dtp['category'].rename('вид ДТП')` — Series с новым именем: имя становится подписью оси категорий.

– `value_counts().head(8)` — те же восемь чисел под графиком.

Ячейка 20. Исход по ролям

```
ax = share_bars(main['role'].rename('роль'), main['health'].rename('исход'), ['Не пострадал', 'Ранен', 'Погиб'],
                'Исход ДТП для участника по ролям')  # состав исходов по ролям
ax.set_ylabel('Доля участников')
ax.legend(title='исход', loc='center left', bbox_to_anchor=(1, 0.5))  # легенда справа
pd.crosstab(main['role'], main['health'], normalize='index').round(3)  # доли исходов по ролям
```

– `main['role'].rename('роль')` и `main['health'].rename('исход')` — имена становятся подписью оси групп и заголовком легенды; `['Не пострадал', 'Ранен', 'Погиб']` — порядок слоёв.

– `ax.set_ylabel('Доля участников')` — функция подписала ось «Доля ДТП».

– `ax.legend(title='исход', loc='center left', bbox_to_anchor=(1, 0.5))` — легенда справа от графика: `bbox_to_anchor=(1, 0.5)` — точка на правом краю осей на половине высоты, `loc='center left'` — к ней прикладывается середина левого края легенды. Легенда строится заново, поэтому заголовок указан ещё раз.

– `pd.crosstab(..., normalize='index').round(3)` — те же доли числами.

Ячейка 21. ДТП по годам

```
count_line(dtp['year'], 'ДТП по годам', 'Год');  # линия по годам
```

– `count_line(dtp['year'], ...)` — линия числа ДТП по годам; точка с запятой — как в ячейке 15.

Ячейка 22. Доля ДТП с погибшими по годам

```
share_line(dtp, 'year', 'fatal', 'Доля ДТП с погибшими по годам', 'Год')  # доля с полосой
dtp.groupby('year')['fatal'].mean().round(3).to_frame('доля ДТП с погибшими').T
```

– `share_line(dtp, 'year', 'fatal', ...)` — доля ДТП с погибшими по годам с полосой.

– `groupby('year')['fatal'].mean()` — доля по годам.

– `to_frame(...)` — Series в таблицу из одного столбца с названием «доля ДТП с погибшими»; `.T` — транспонирование: годы становятся столбцами, и таблица занимает одну строку.

Ячейка 23. Доля ДТП с погибшими по освещению

```
short = {'Светлое время суток': 'день', 'Сумерки': 'сумерки', 'В темное время суток, освещение включено': 'темно, свет включён',
         'В темное время суток, освещение не включено': 'темно, свет не включён',
         'В темное время суток, освещение отсутствует': 'темно, освещения нет'}  # короткие подписи освещения
lit = dtp.assign(lighting=dtp['light'].map(short)).sort_values('lighting')  # освещение коротко
fig, ax = plt.subplots(figsize=(10, 4))  # шире для подписей
mean_bars(lit, 'lighting', 'fatal', 'Доля ДТП с погибшими при разном освещении', 'Освещение', 'Доля ДТП с погибшими', ax=ax)
ax.tick_params(axis='x', labelsize=9)  # длинные подписи мельче
lit.groupby('lighting')['fatal'].agg(['mean', 'count']).round(3)  # доля и число ДТП
```

– Строки 1–3 — словарь коротких подписей: ключ — значение из данных, значение — короткая подпись. В данных «темное» записано через «е».

– `dtp['light'].map(short)` — каждое значение заменяется по словарю; `assign` — новый столбец `lighting` в копии таблицы.

– `sort_values('lighting')` — строки по алфавиту подписей: столбцы графика идут в порядке, в котором значения встречаются в таблице.

– `fig, ax = plt.subplots(figsize=(10, 4))` — свои оси шире обычных; `mean_bars(..., ax=ax)` рисует на них.

– `ax.tick_params(axis='x', labelsize=9)` — подписи оси X мельче.

– `agg(['mean', 'count'])` — доля и число ДТП при каждом освещении.

Ячейка 24. Час ДТП по тяжести, три панели

```
known = dtp[dtp['severity'] != 'Без пострадавших (нет данных)']  # три тяжести
fig = hist_panels(known, 'hour', 'severity', 'Час ДТП по тяжести')
for ax in fig.axes:
    ax.set_title(ax.get_title().replace('severity = ', ''))  # тяжесть в заголовке панели
    ax.set_xlabel('Час')
fig.axes[0].set_ylabel('Число ДТП')
fig.subplots_adjust(top=0.8);  # место для общего заголовка
```

– Строка 1 — три тяжести: строки со служебным значением «Без пострадавших (нет данных)» отброшены.

– `hist_panels(known, 'hour', 'severity', ...)` — панели часа по тяжести; функция возвращает фигуру.

– `fig.axes` — список осей всех панелей; цикл правит каждую.

– `ax.get_title().replace('severity = ', '')` — seaborn подписывает панель «severity = » и тяжесть, в заголовке остаётся только тяжесть.

– `fig.axes[0].set_ylabel('Число ДТП')` — подпись оси Y у первой панели слева.

– `fig.subplots_adjust(top=0.8)` — панели занимают фигуру до 80 процентов высоты, сверху остаётся место для общего заголовка.

Ячейка 25. Доля ночных ДТП по тяжести

```
known = known.assign(night=known['hour'].isin([22, 23, 0, 1, 2, 3, 4, 5]))  # с 22:00 до 5:59
mean_bars(known, 'severity', 'night', 'Доля ночных ДТП по тяжести', 'Тяжесть ДТП', 'Доля ДТП с 22:00 до 5:59')
known.groupby('severity')['night'].agg(['mean', 'count']).round(3)  # доля и число ДТП
```

– `known['hour'].isin([22, 23, 0, 1, 2, 3, 4, 5])` — флаг ночного ДТП, с 22:00 до 5:59; `assign(night=...)` — новый столбец.

– `mean_bars(known, 'severity', 'night', ...)` — доля ночных ДТП по тяжести с усами интервала.

Ячейка 26. Карта ДТП с погибшими

```
box = (51.8, 54.2, 31, 35.5)  # рамка области
in_box = lit['lat'].between(51.8, 54.2) & lit['lon'].between(31, 35.5)  # ДТП внутри рамки
print((lit['fatal'] & in_box).sum(), lit['fatal'].sum())
fig = points_map(lit[lit['fatal']], box, 'lighting', ['district', 'datetime', 'category'], 'ДТП с погибшими')  # карта ДТП с погибшими
fig
```

– `box` — рамка области: широта от 51.8 до 54.2, долгота от 31 до 35.5.

– `in_box` — маска ДТП внутри рамки; строка 3 — ДТП с погибшими внутри рамки и все ДТП с погибшими.

– `points_map(lit[lit['fatal']], box, 'lighting', [...], ...)` — карта ДТП с погибшими: цвет — освещение, в подсказке — район, дата и вид ДТП.

– `fig` последней строкой — показать карту.

7 Аномалии

Ячейка 27. Выбросы по трём правилам

```
print(iqr_bounds(dtp['dead_count']), iqr_bounds(drivers['experience_clean']), iqr_bounds(drivers['vehicle_age']))  # границы IQR
pd.concat([outlier_table(dtp, numeric), outlier_table(drivers, ['experience_clean', 'vehicle_age'])])  # выбросы по трём правилам
```

– Строка 1 — границы IQR трёх столбцов: каждая — пара чисел в скобках.

– `pd.concat([..., ...])` — две таблицы выбросов одна под другой: счётчики ДТП и столбцы водителей взяты из разных таблиц.

Ячейка 28. Самое тяжёлое ДТП

```
dtp.loc[dtp['dead_count'].idxmax(), ['datetime', 'district', 'category', 'participants_count', 'dead_count', 'injured_count']]  # самое тяжёлое ДТП
```

– `dtp['dead_count'].idxmax()` — номер ДТП с наибольшим числом погибших, как на семинаре 10.

– `dtp.loc[..., [...]]` — одна строка и шесть столбцов; результат — Series.

Ячейка 29. Логические проверки

```
checks = logic_checks(dtp)  # четыре проверки счётчиков
inside = dtp['lat'].between(51.8, 54.2) & dtp['lon'].between(31, 35.5)  # координаты в рамке
checks['координаты вне рамки области'] = (~inside).sum()
checks['нулевые координаты'] = ((dtp['lat'] == 0) & (dtp['lon'] == 0)).sum()
checks['год выпуска раньше 1900'] = (people['vehicle_year'] < 1900).sum()
checks['из них у водителей'] = ((people['vehicle_year'] < 1900) & (people['role'] == 'Водитель')).sum()
checks['дата вне 2015-2025 годов'] = (~dtp['year'].between(2015, 2025)).sum()
rows_per_dtp = people['dtp_id'].value_counts().reindex(dtp.index)  # строк участников на ДТП
checks['участников не столько, сколько строк'] = (rows_per_dtp != dtp['participants_count']).sum()
checks
```

– `checks = logic_checks(dtp)` — Series четырёх проверок; `checks['...'] = ...` — присваивание по новой метке добавляет проверку.

– `~inside` — отрицание маски: ДТП вне рамки.

– Строки 5–6 — год выпуска раньше 1900 у всех участников и у водителей.

– `between(2015, 2025)` — год внутри периода, края включены.

– `people['dtp_id'].value_counts()` — строк участников на каждый номер ДТП; `reindex(dtp.index)` — в порядке таблицы ДТП, номер без строк участников дал бы пропуск.

– `rows_per_dtp != dtp['participants_count']` — сравнение по номерам ДТП: у обеих Series один и тот же индекс.

Ячейка 30. Повторы

```
print(dtp.duplicated().sum(), dtp.index.duplicated().sum())  # повторы ДТП и номеров
duplicate_report(people, 'role').head(4)  # повторы участников по ролям
```

– `dtp.duplicated().sum()` — повторы строк, `dtp.index.duplicated().sum()` — повторы номера ДТП в индексе.

– `duplicate_report(people, 'role').head(4)` — четыре роли с наибольшим числом строк в повторах.

8 Проверка гипотез

Ячейка 31. Нормальность и выбор теста

```
normality_report(drivers, ['experience_clean', 'vehicle_age'])  # нормальность и выбор теста
```

– `normality_report(drivers, [...])` — таблица нормальности стажа и возраста машины со столбцом «что дальше».

Ячейка 32. Q–Q графики

```
qq_panels(drivers, ['experience_clean', 'vehicle_age']);  # Q-Q графики
```

– `qq_panels(...)` — два Q–Q графика рядом; точка с запятой — как в ячейке 15.

Ячейка 33. Сводная таблица шести гипотез

```
hypotheses_report(dtp, drivers).round(3)  # шесть гипотез с поправкой
```

– `hypotheses_report(dtp, drivers).round(3)` — шесть гипотез с решением при 0.05 и после поправки, числа с тремя знаками.

Ячейка 34. Тёмное время и ДТП с погибшими

```
flags = dtp.assign(dark=np.where(dtp['light'].str.contains('темное'), 'тёмное время', 'светлое время и сумерки'))  # время суток словами
print(pd.crosstab(flags['dark'], flags['fatal'], margins=True, margins_name='всего'))  # счётчики ДТП
chi, p, dof, v = chi_square(flags, 'dark', 'fatal')  # хи-квадрат и V Крамера
print(round(chi, 2), p, dof, round(v, 3), effect_label(v, 'V'))
mean_bars(flags.sort_values('dark'), 'dark', 'fatal', 'Доля ДТП с погибшими в тёмное и светлое время', 'Время суток', 'Доля ДТП с погибшими')
flags.groupby('dark')['fatal'].mean().round(3)  # доли по строкам
```

– Строка 1 — `np.where(dtp['light'].str.contains('темное'), ...)`: столбец `dark` словами, а не `True` и `False`, чтобы подписи сразу читались в таблице и на графике. В данных «темное» записано через «е», а в подписи — через «ё».

– Строка 2 — таблица счётчиков с итогами, как в ячейке 14.

– Строки 3–4 — `chi_square` возвращает четыре числа: статистику, p, число степеней свободы и коэффициент V Крамера; `effect_label(v, 'V')` — размер связи словом.

– Строка 5 — `flags.sort_values('dark')`: столбцы графика по алфавиту подписей.

– Строка 6 — доли ДТП с погибшими по строкам.

Ячейка 35. Стаж женщин и мужчин за рулём

```
women = drivers.loc[drivers['gender'] == 'Женский', 'experience_clean'].dropna()  # стаж женщин за рулём
men = drivers.loc[drivers['gender'] == 'Мужской', 'experience_clean'].dropna()  # стаж мужчин за рулём
t, p, d = t_two(women, men)  # t Уэлча и d Коэна
weighted = np.sqrt(((len(women) - 1) * women.var() + (len(men) - 1) * men.var()) / (len(women) + len(men) - 2))  # отклонение с весами групп
print(len(women), len(men), round(t, 2), p, round(d, 3), round((women.mean() - men.mean()) / weighted, 3))
print(round(women.mean(), 2), round(men.mean(), 2), round(women.std(), 2), round(men.std(), 2))  # средние и разброс
print(women.quantile([0.25, 0.5, 0.75]).tolist(), men.quantile([0.25, 0.5, 0.75]).tolist())  # квартили и медианы
low_w, high_w = ci_mean(women)  # интервал среднего женщин
low_m, high_m = ci_mean(men)  # интервал среднего мужчин
print(round(low_w, 2), round(high_w, 2), round(low_m, 2), round(high_m, 2))
ax = box_by(drivers, 'experience_clean', 'gender', ['Женский', 'Мужской'], 'Стаж водителей по полу')  # ящики двух групп
ax.set_xlabel('Пол водителя')
ax.set_ylabel('Стаж, лет');
```

– Строки 1–2 — `drivers.loc[маска, 'experience_clean'].dropna()`: стаж женщин и стаж мужчин без пропусков.

– Строка 3 — t-тест Уэлча и коэффициент d Коэна, распаковка в три переменные.

– Строка 4 — классическое объединённое отклонение: дисперсии групп с весами `len(women) - 1` и `len(men) - 1`, сумма делится на `len(women) + len(men) - 2`, и из результата берётся корень. Строка 5 печатает и d по этой формуле.

– Строки 6–7 — средние, стандартные отклонения и квартили; `tolist()` — Series в обычный список, так печать короче.

– Строки 8–10 — `ci_mean` возвращает две границы, их раскладывают в две переменные.

– Строки 11–13 — ящики двух групп и подписи осей.